

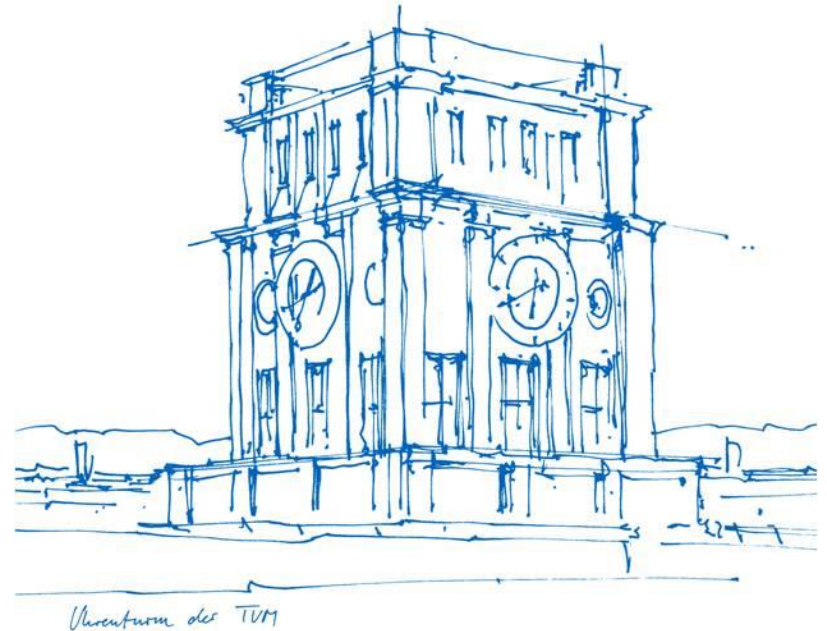
Basics of FIM: Statistical Learning

6: Neural Networks

Univ.-Prof. Dr-Ing. habil. Niklas Kühl

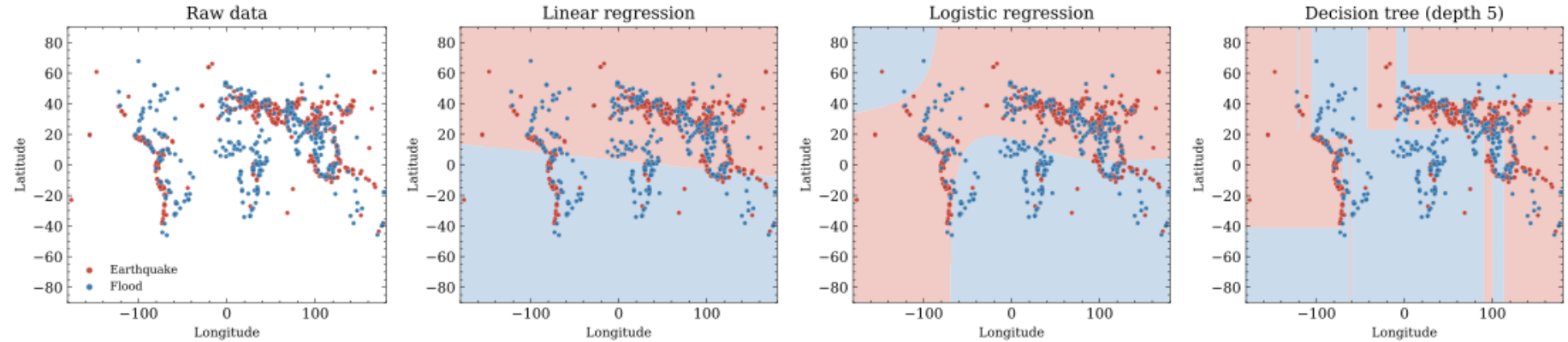
Lars Böcking

Chair for Information Systems and Human-centric
Artificial Intelligence
University of Bayreuth
TUM School of Management



A straight line underfits; disaster patterns are nonlinear.

On the disaster data (Earthquake vs. Flood by longitude/latitude): linear draws a straight boundary, logistic a curved one, a tree blocky regions.



We want to learn a smooth, nonlinear decision boundary

- Motivation to Neural Networks
- Network architecture
- Loss and backpropagation
- Optimization
- Regularization and tuning

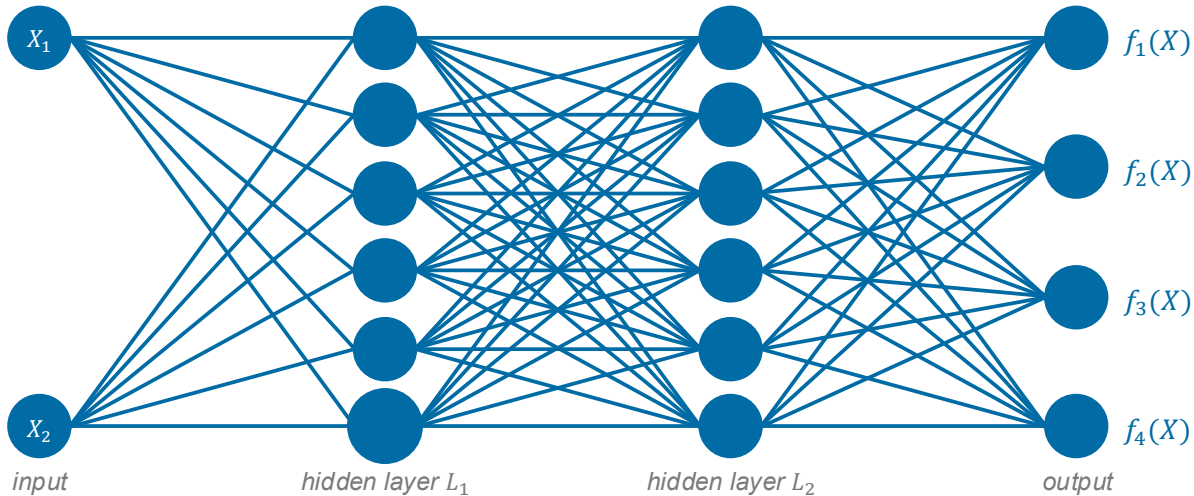
A neural network stacks an input, hidden, and output layer.



Inputs $X = (X_1, \dots, X_p)$ feed the *brain*.

The output layer produces $f(X)$, where the index indicates the different target classes in case of classification.

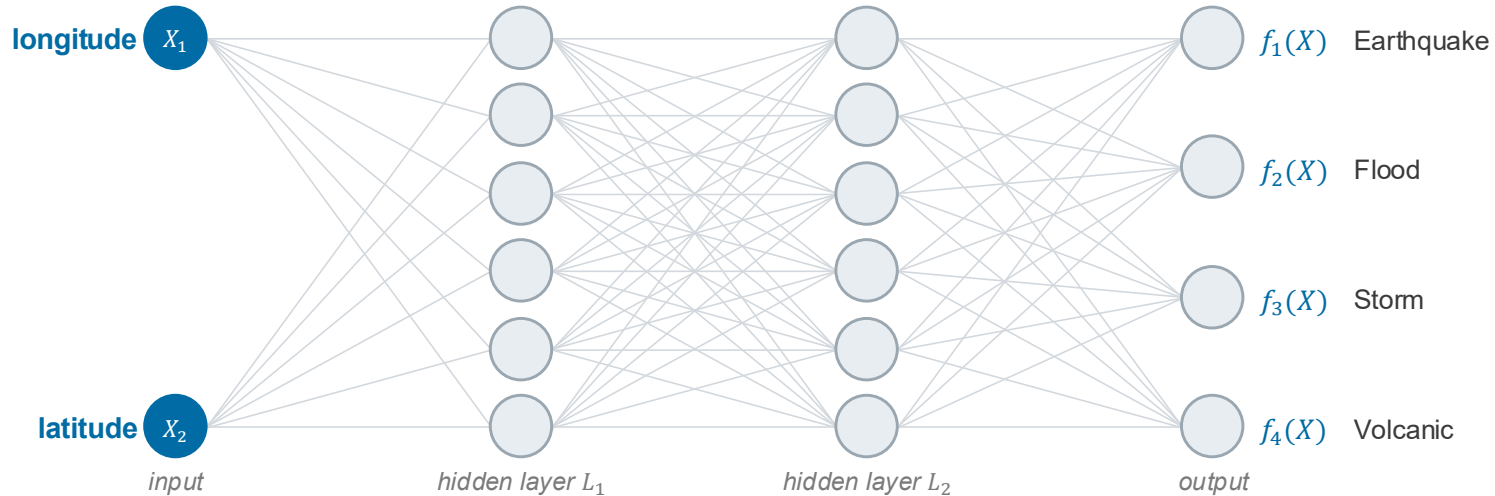
A neural network stacks an input, hidden, and output layer.



Inputs $X = (X_1, \dots, X_p)$ feed the hidden layers.
The output layer produces $f(X)$, where the index indicates the different target classes in case of classification.

- Motivation to Neural Networks
- **Network architecture**
- Loss and backpropagation
- Optimization
- Regularization and tuning

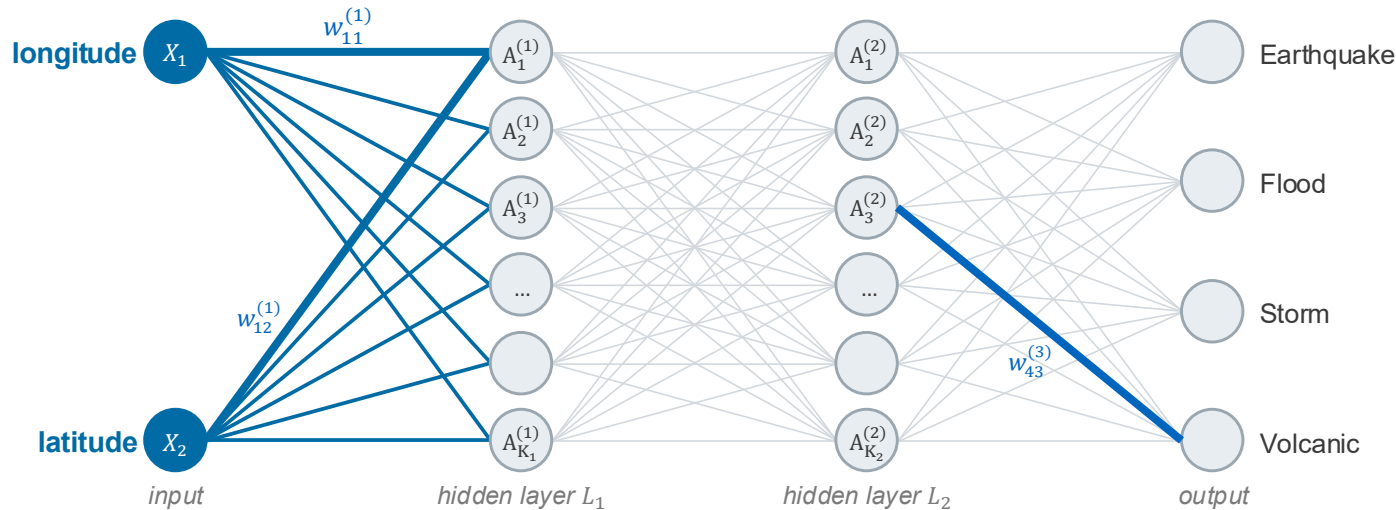
The input layer holds the p features we feed the network.



$$X = (X_1, X_2)$$

here X_1 = longitude, X_2 = latitude and $f_1(X)$ = Earthquake etc.

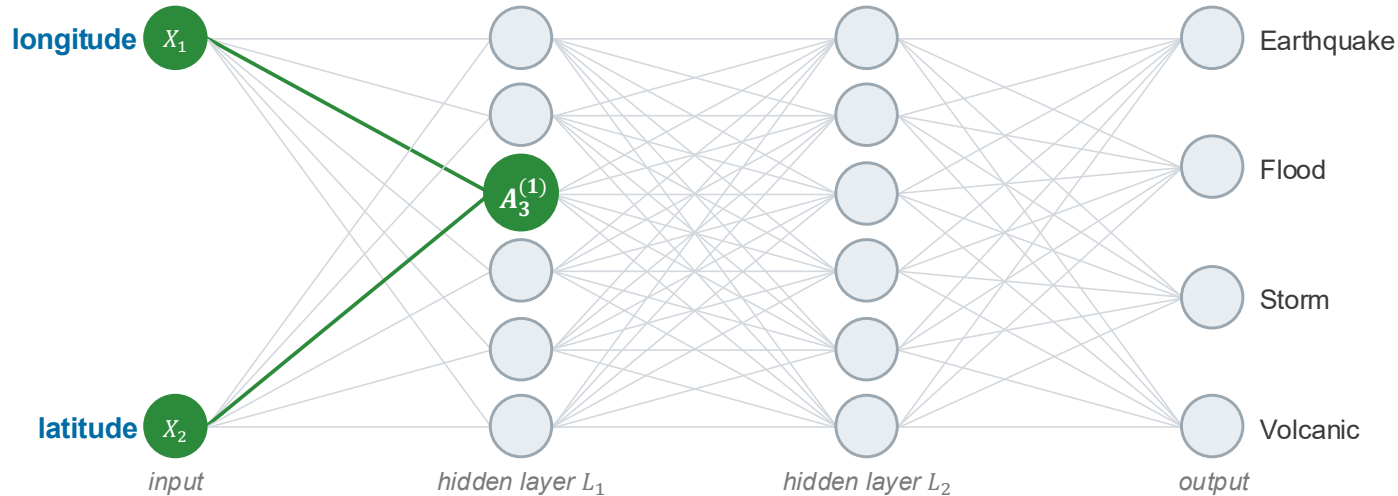
Weights on the edges scale every signal that flows forward.



$w_{kj}^{(l)}$ is the weight that connects the j neuron of the $l - 1$ layer to the k neuron of the l layer.
The activations $A_k^{(l)}$ is the result of a hidden unit $h_k^{(l)}$ processing its input, e.g. $A_k^{(1)} = h_k^{(1)}(X)$.

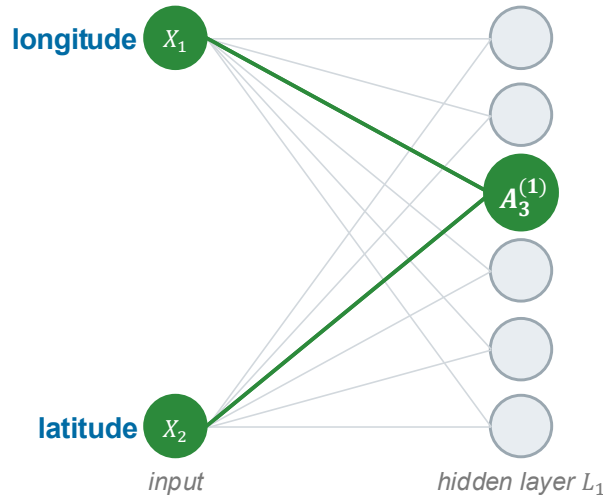
The notation might seem counterintuitive (at least for me it did), but we follow the definition from book p. 404

Each layer treats the activations of the previous layer as inputs.



We get the first layer's activation $A_k^{(1)}$ by passing the input X to the hidden layer $h_k^{(1)}$.

A nonlinear activation g makes the network more than a line.



We get the first layer's activation $A_k^{(1)}$ by passing the input X to the hidden layer $h_k^{(1)}$.

A hidden unit $h_k^{(l)}$ is a combination of a linear processing of the previous layer's output:

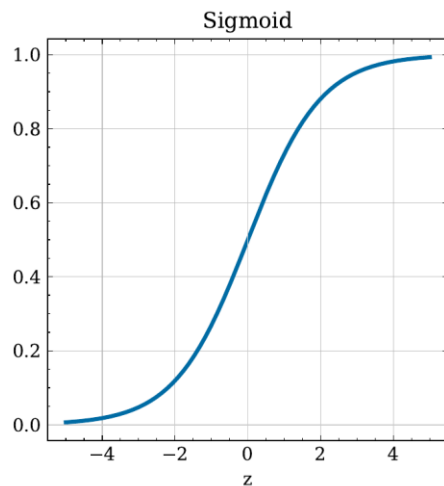
$$z_k^{(l)} = w_{k0}^{(l)} + \sum_{j=1}^{K_{l-1}} w_{kj}^{(l)} A_j^{(l-1)}$$

and a non-linear transformation function g , such that:

$$A_k^{(l)} = h_k^{(l)}(A^{(l-1)}) = g(z_k^{(l)}).$$

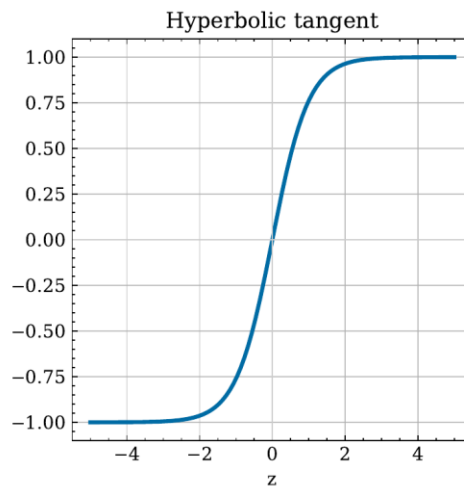
Without g , $f(X)$ would collapse into a linear model in X .

Activation functions provide non-linear transformations.



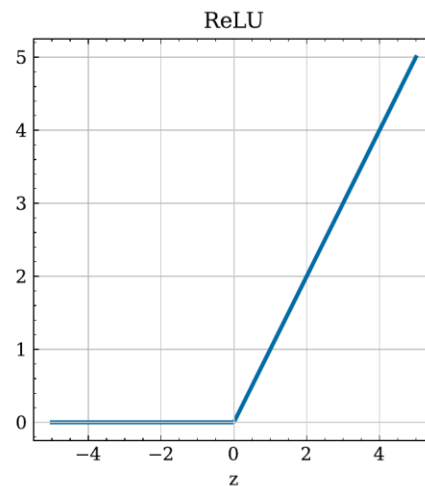
$$g(z) = \frac{e^z}{1 + e^z}$$

sigmoid



$$g(z) = \tanh(z)$$

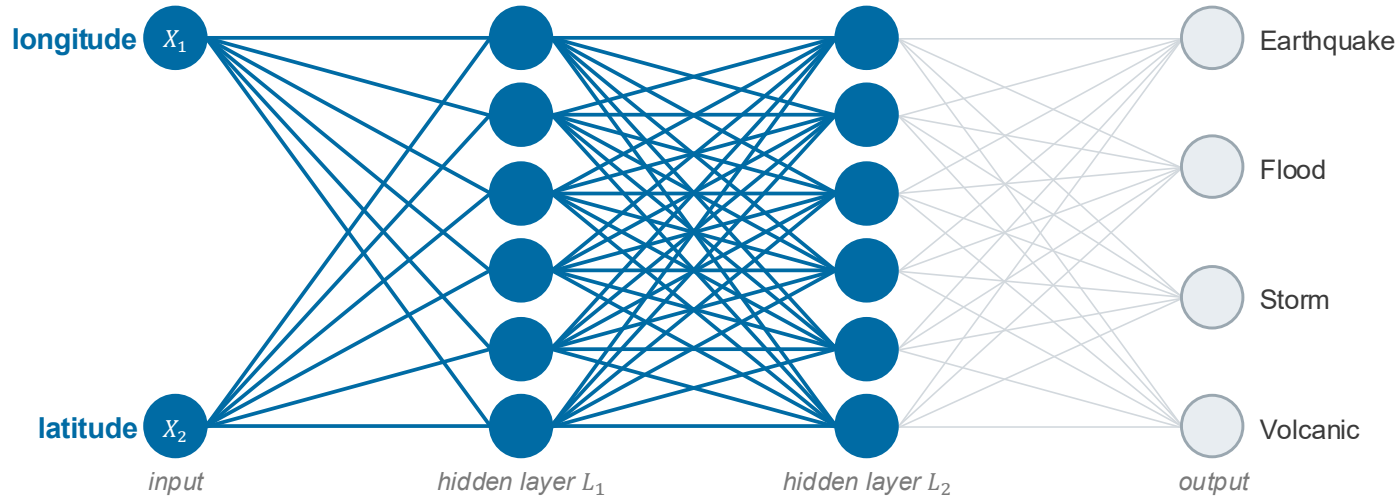
tanh



$$g(z) = (z)_+ = \max(0, z)$$

ReLU

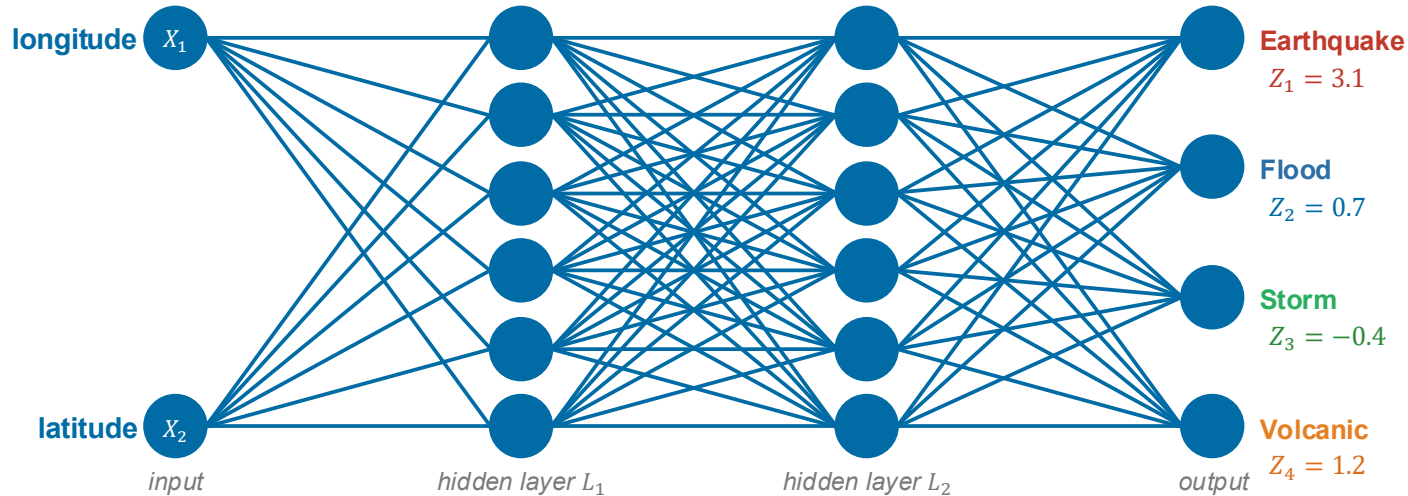
Layer by layer: the multilayer forward pass.



We continue this calculation throughout the network:

$$A_k^{(l)} = h_k^{(l)}(A^{(l-1)}) = g\left(w_{k0}^{(l)} + \sum_{j=1}^{K_{l-1}} w_{kj}^{(l)} A_j^{(l-1)}\right)$$

The output layer computes one linear score Z_m per class.



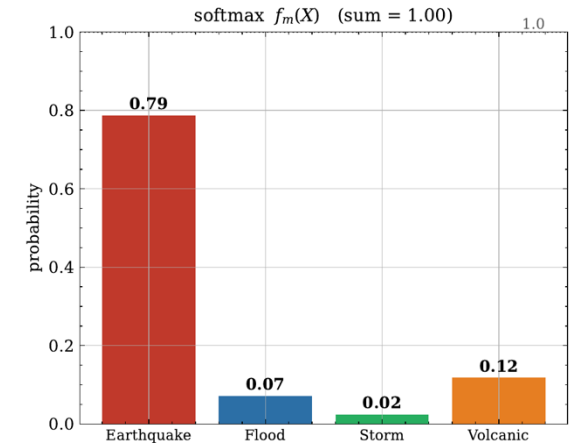
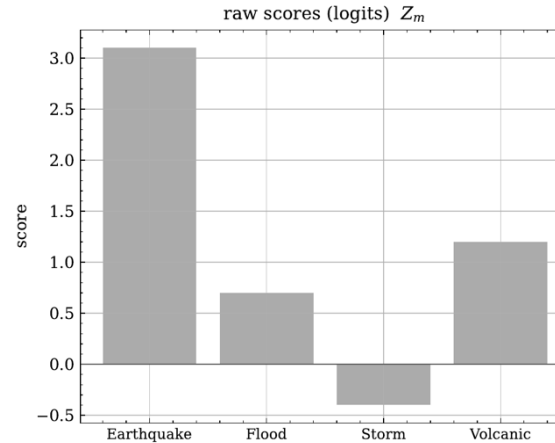
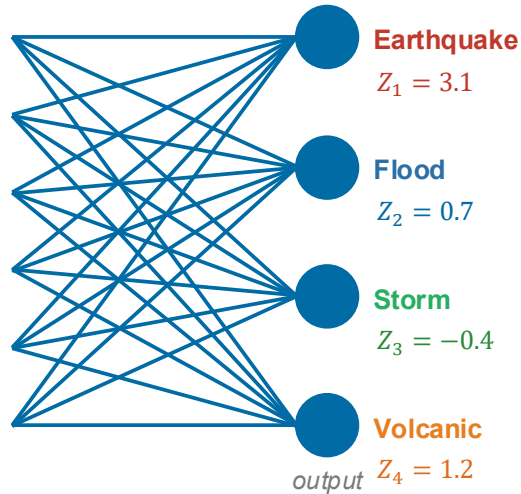
We continue this calculation throughout the network:

$$A_k^{(l)} = h_k^{(l)}(A^{(l-1)}) = g\left(w_{k0}^{(l)} + \sum_{j=1}^{K_{l-1}} w_{kj}^{(l)} A_j^{(l-1)}\right)$$

... until we get to the final layer.

Where we stop before applying the nonlinear transformation g . This output is referred to as **logits Z_m** . We want the final output to represent class probabilities.

Softmax turns the logits Z_m into class probabilities $f_m(X)$.

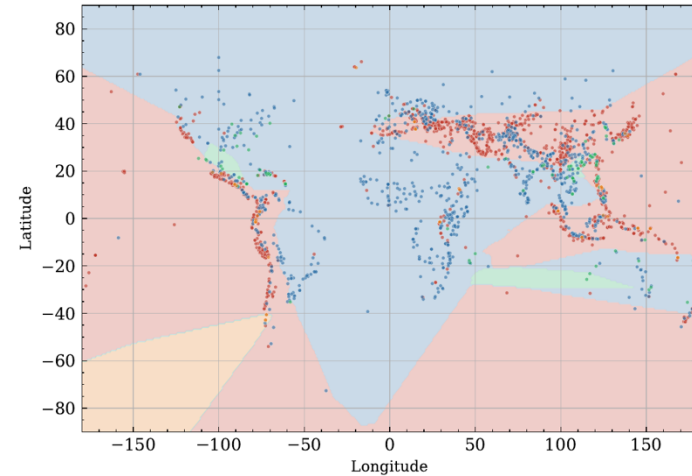
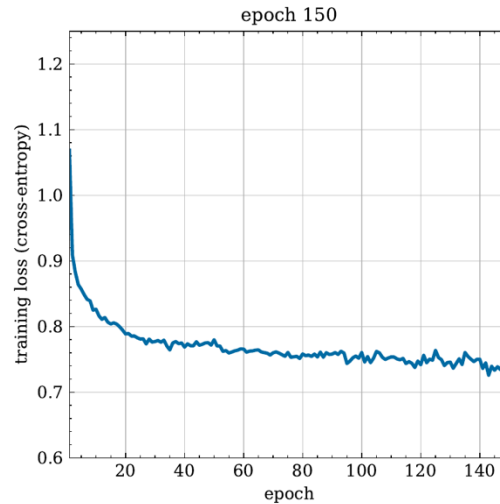


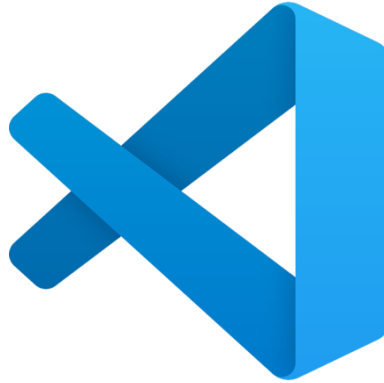
Where we stop before applying the non linear transformation g . This output is referred to as **logits** Z_m . We want the final output to represent class probabilities.

Just like in multinomial logistic regression we use softmax activation function:

$$\text{softmax}(Z_m) = \frac{e^{Z_m}}{\sum_m e^{Z_m}}$$

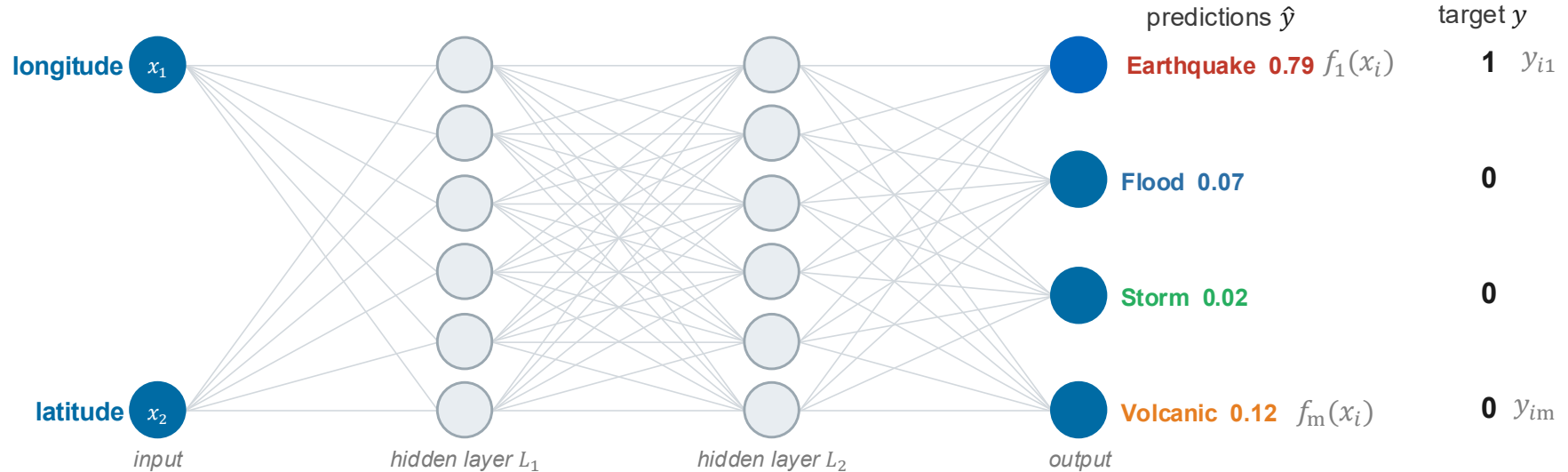
Trained end-to-end, the network separates disaster types.





- Motivation to Neural Networks
- Network architecture
- **Loss and backpropagation**
- Optimization
- Regularization and tuning

The loss compares the predicted vector with the target.



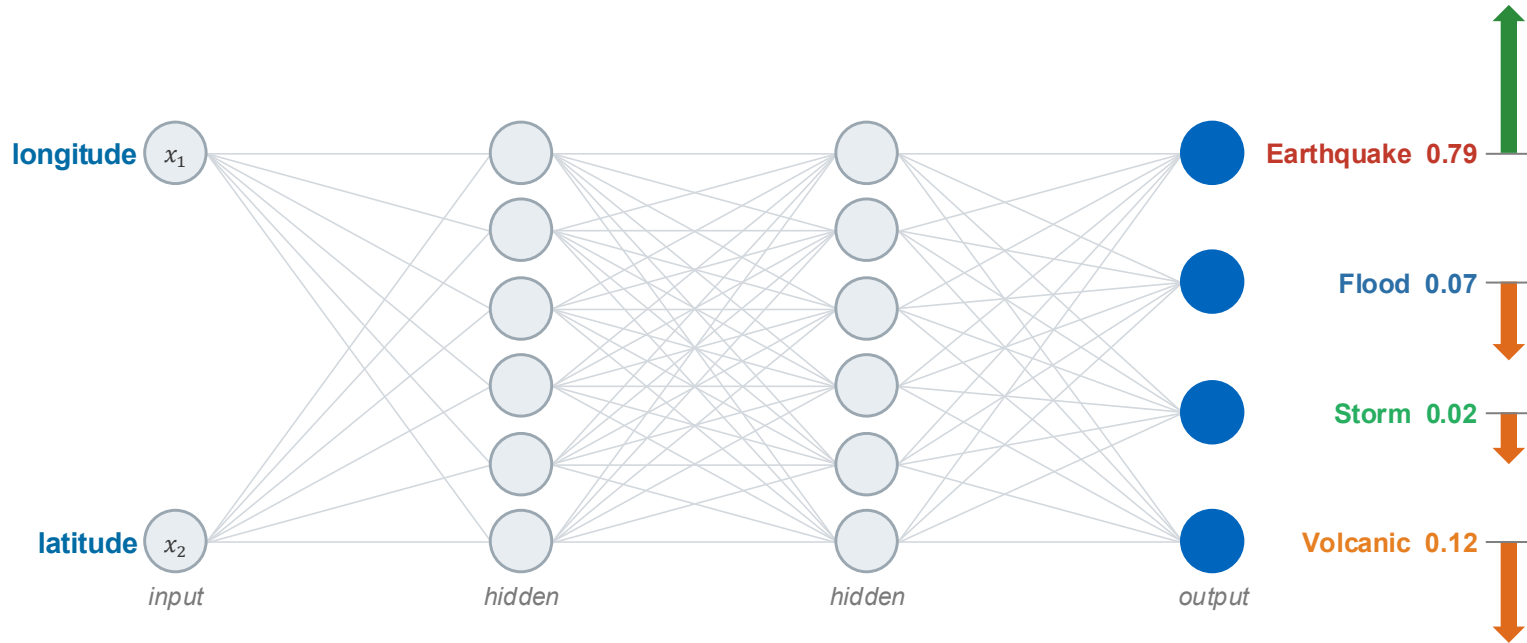
We put all weights $w_{kj}^{(l)}$ into a vector θ and define our objective (**cross-entropy**) to be:

$$\min_{\theta} R(\theta) = - \sum_{i=1}^n \sum_{m=1}^M y_{im} \log(f_m(x_i))$$

For a quantitative response we use the **squared-error loss**:

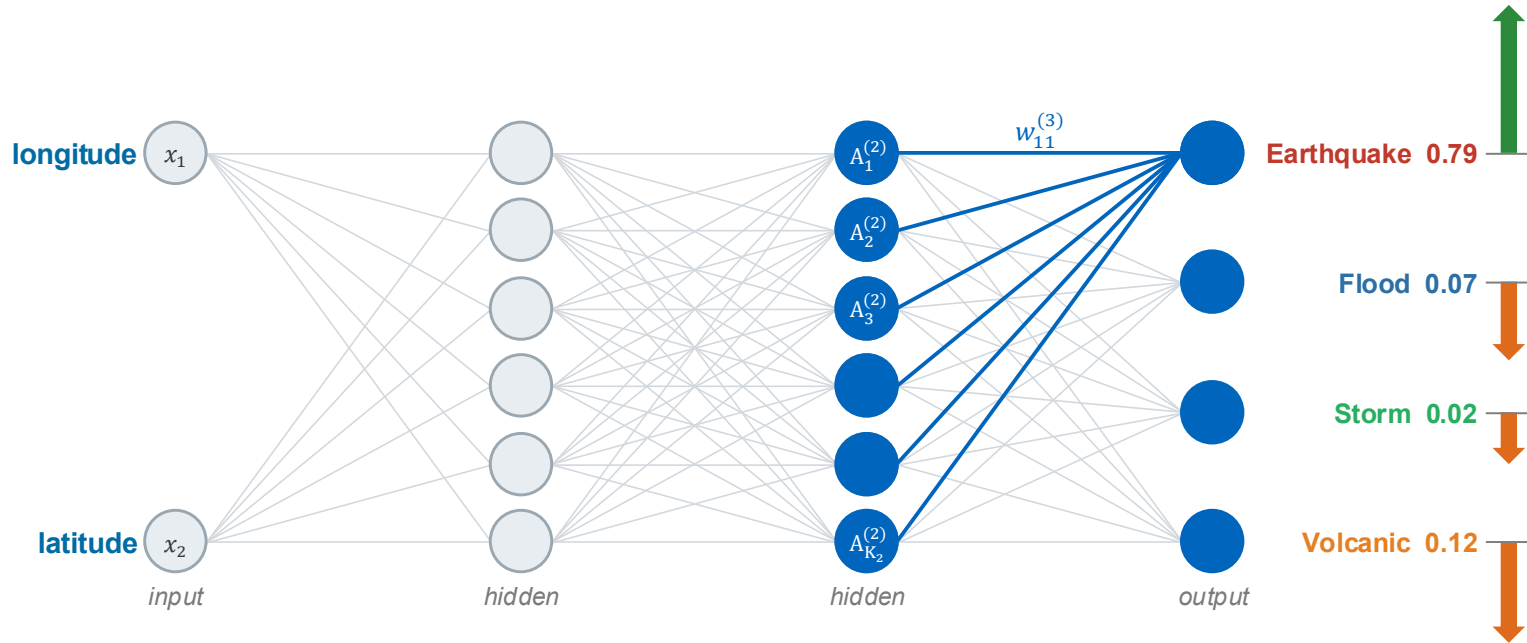
$$R(\theta) = \sum_{i=1}^n (y_i - f_{\theta}(x_i))^2$$

We can derive which output should be larger or smaller.



For this example the target is Earthquake. We want $\hat{y}_1$ **larger** and the other three **smaller**.
Arrow size = how far each output was off.

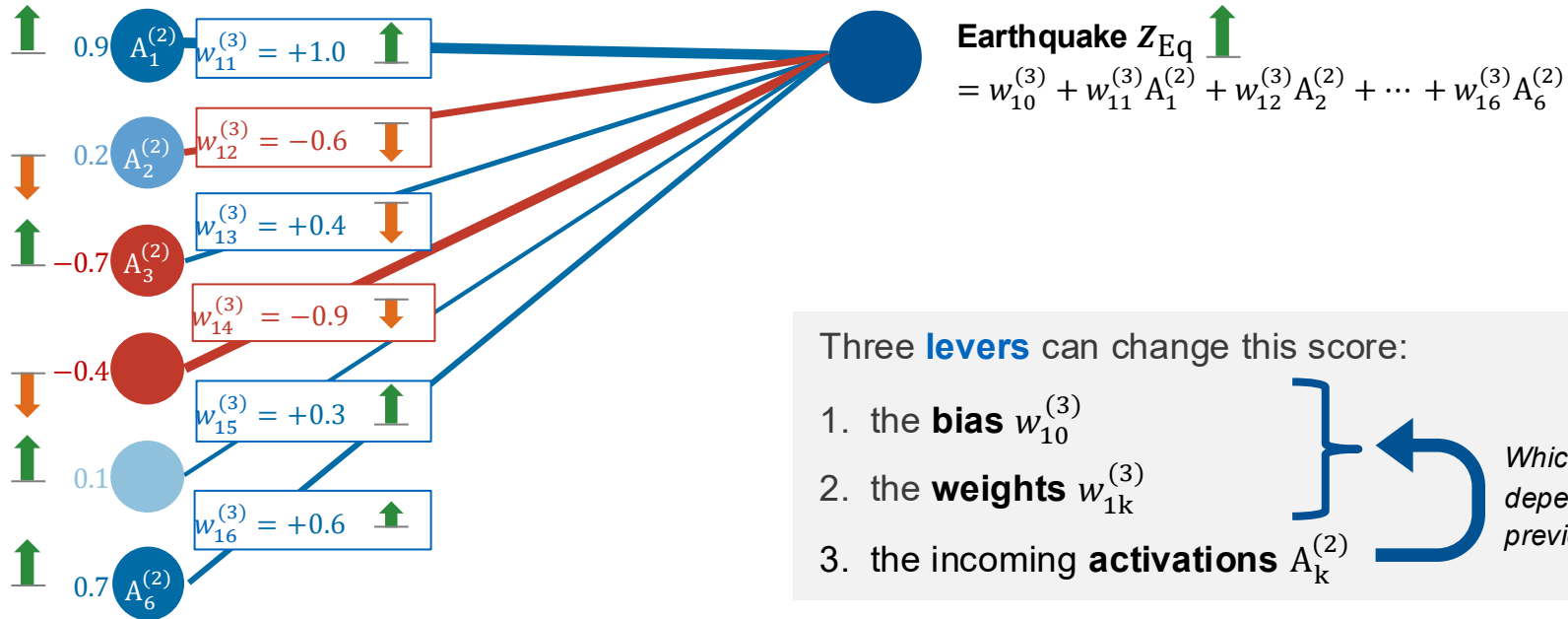
Each parameter contributed differently to the current prediction.



Every parameter in θ has a direction that reduces this example's error $R_i(\theta)$.

Zoom in on the neuron and its incoming weights.

Every parameter in θ has a direction that reduces this example's error $R_i(\theta)$.



Three **levers** can change this score:

1. the **bias** $w_{10}^{(3)}$
2. the **weights** $w_{1k}^{(3)}$
3. the incoming **activations** $A_k^{(2)}$

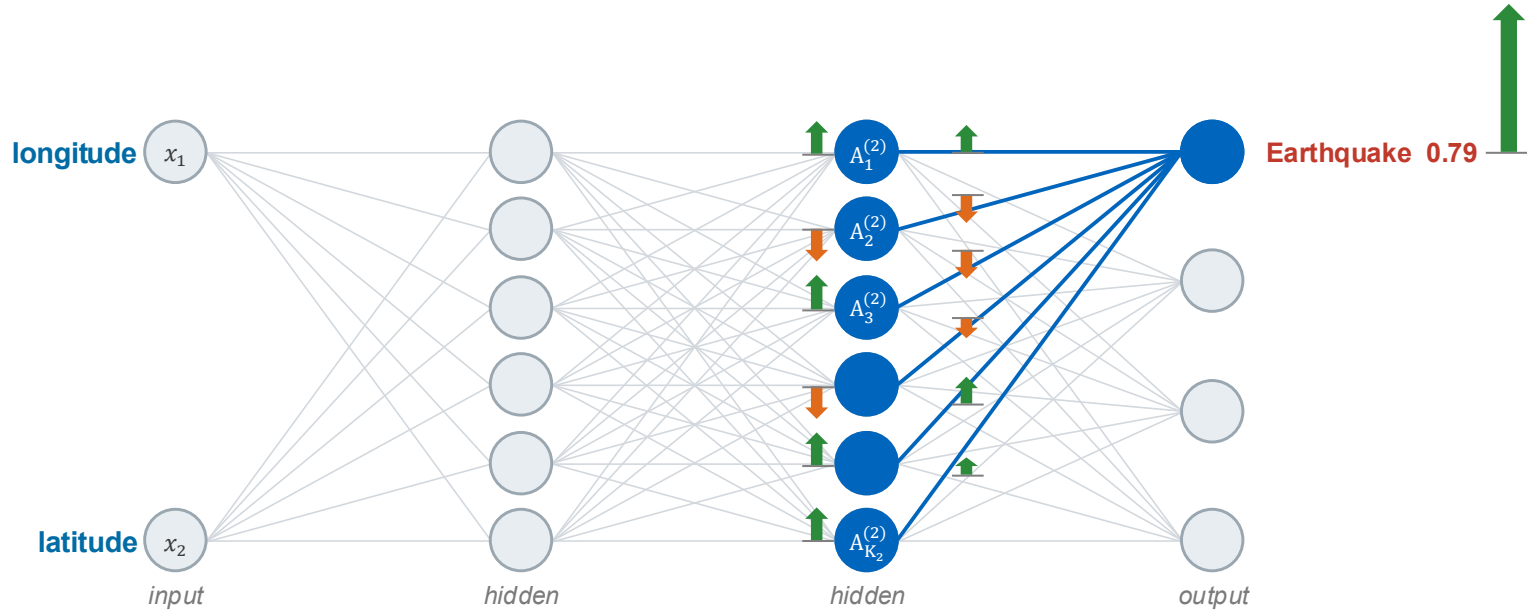
Which in turn only depends on the previous layers ...

blue = positive

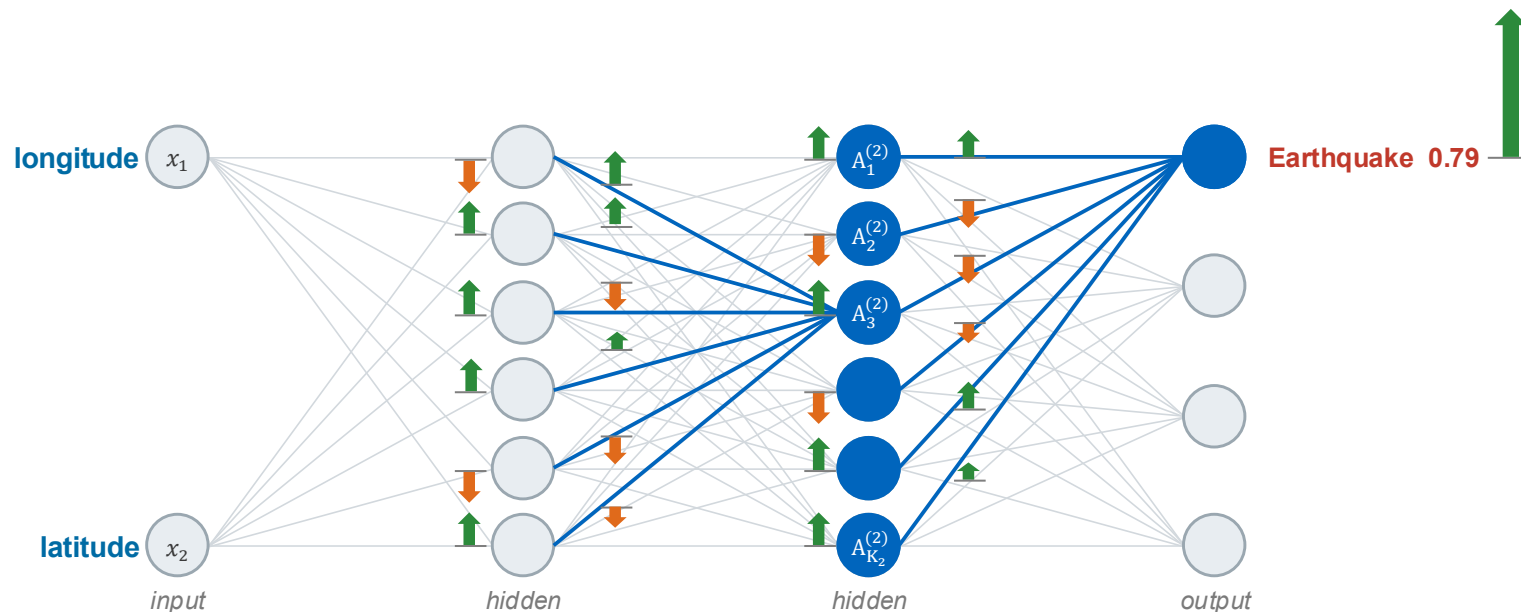
red = negative

thickness = size of weights

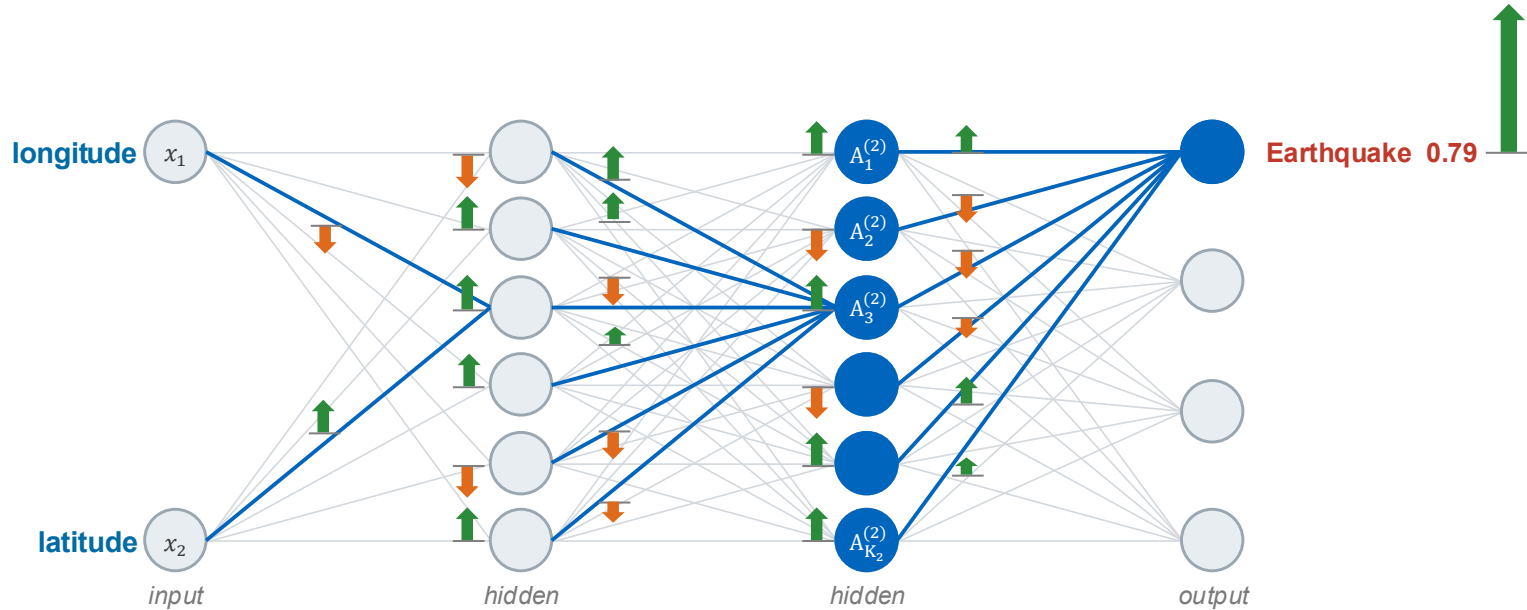
To learn, we nudge the weights toward the target.



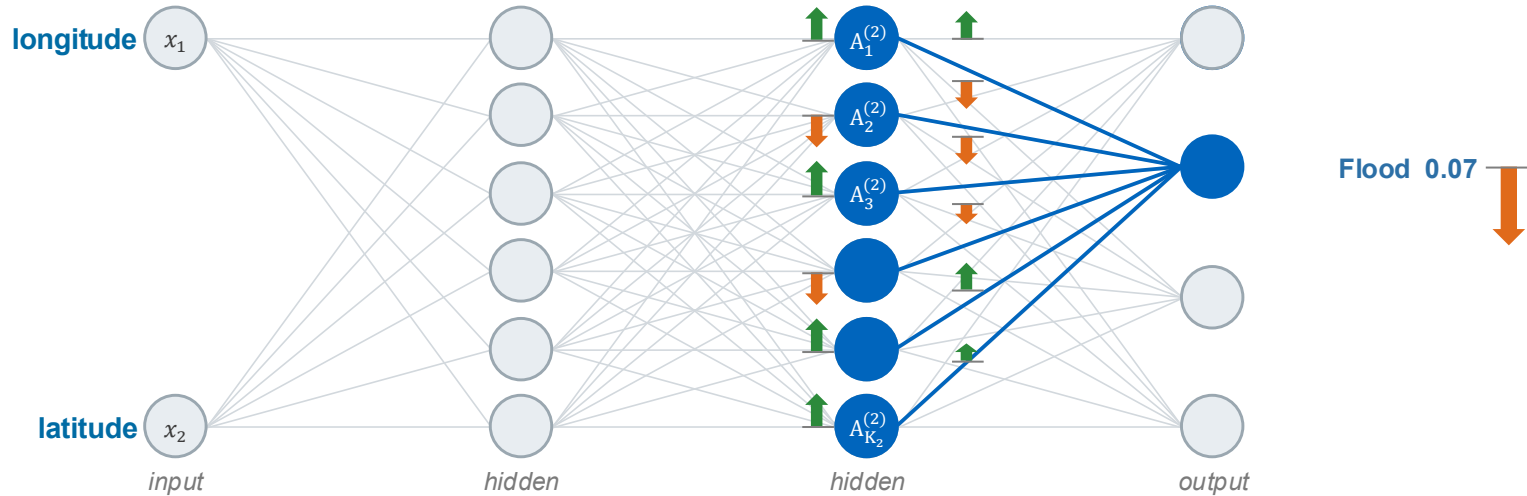
We can backpropagate the changes we would like to see.



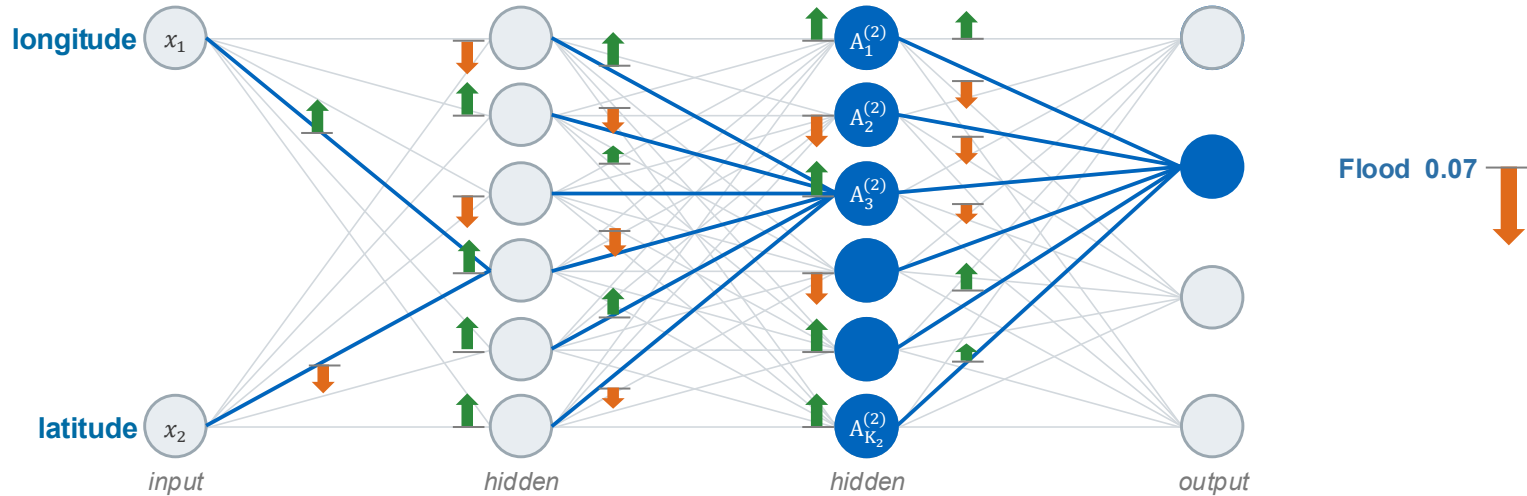
This backpropagation goes all the way to the first weights in our network.



For each output neuron we get different nudges.

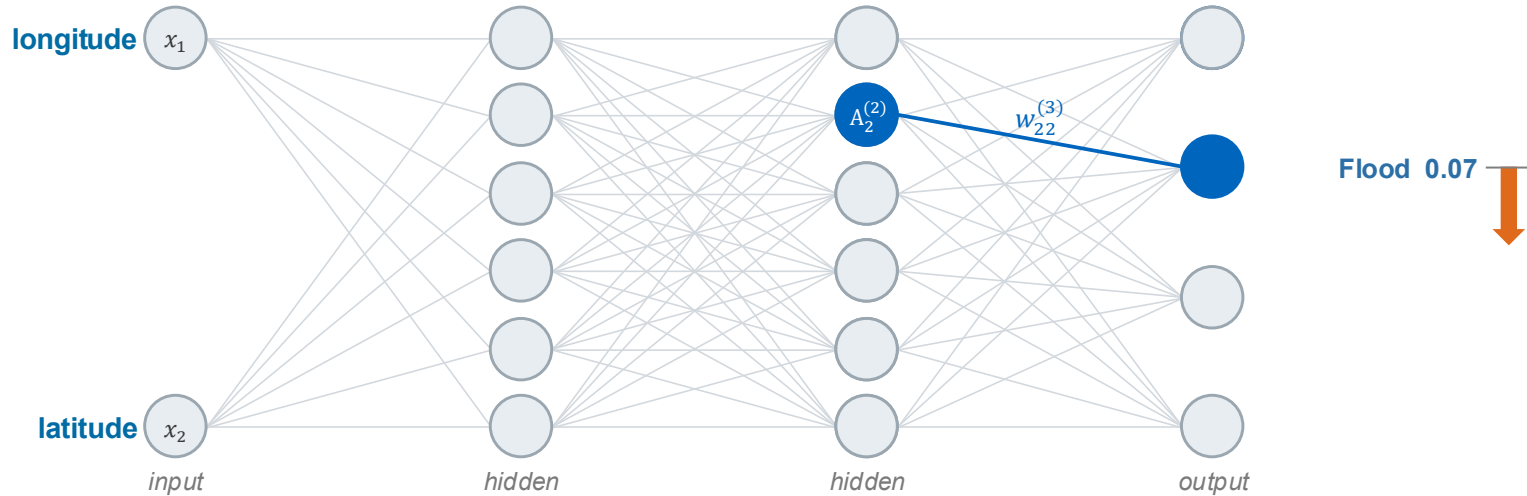


And each of these nudges propagate backwards.



How do we tell a machine how every parameter in θ should change?

We look at a single weight to understand how we adapt it.



How do we tell a machine how every parameter in θ should change?

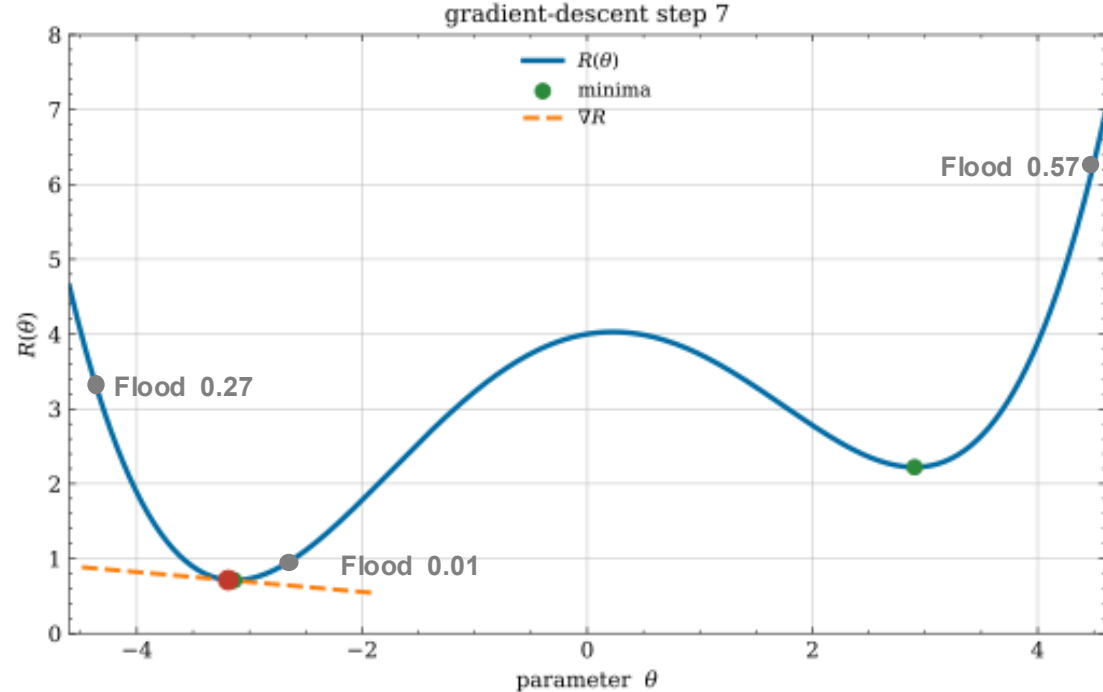
We want to get to the minimum loss.

At the current weight θ^t we read the **tangent**, the gradient of the loss.

$$\nabla R_i(\theta^t)$$

We step downhill by the **learning rate** ρ , then move to the next data point:

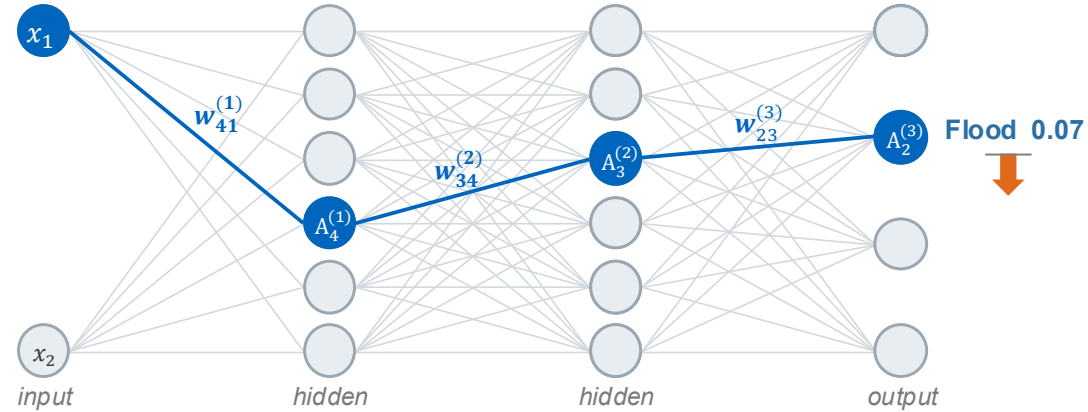
$$\theta^{t+1} \leftarrow \theta^t - \rho \nabla R_i(\theta^t)$$



We follow this gradient across all weights of our network.

At the current weight θ^t we read the **tangent**, the gradient of the loss.

$$\nabla R_i(\theta^t) = \begin{bmatrix} \vdots \\ \nabla w^{(1)} \\ \vdots \\ \nabla w^{(2)} \\ \vdots \\ \nabla w^{(3)} \\ \vdots \end{bmatrix} \begin{matrix} \uparrow \\ \downarrow \\ \uparrow \end{matrix}$$



We step downhill by **the learning rate ρ** , then move to the next data point:

$$\theta^{t+1} \leftarrow \theta^t - \rho \nabla R_i(\theta^t)$$



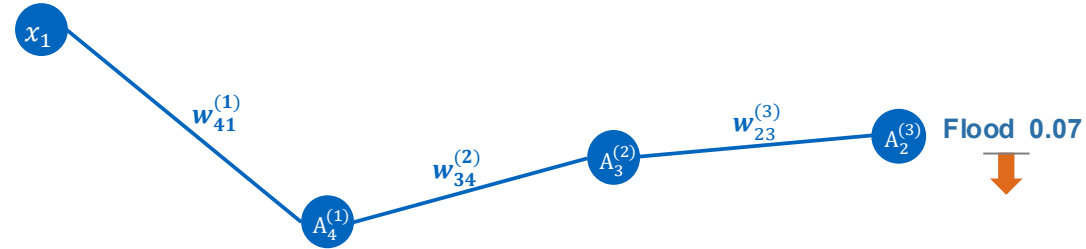
How do we tell a machine how every parameter in θ should change?

We observe the gradient along a simple network.

At the current weight θ^t we read the **tangent**, the gradient of the loss.

$$\nabla R_i(\theta^t)$$

For simplicity, let's assume this to be our network:



By this assumption our cross-entropy loss boils down to:

$$R_i(\theta^t) = -[y_i \log A^{(L)} + (1 - y_i) \log(1 - A^{(L)})]$$

And identify the impact of each parameter.

At the current weight θ^t we read the **tangent**, the gradient of the loss.

$$\nabla R_i(\theta^t) = \left. \frac{\partial R_i(\theta)}{\partial \theta} \right|_{\theta=\theta^t}$$

Gradient is just the partial derivative in each dimension of our parameters.

Start by asking:

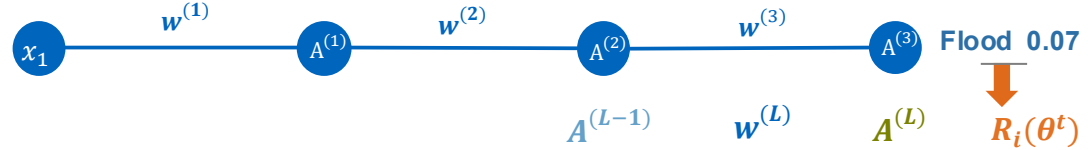
$$\frac{\partial R_i(\theta^t)}{\partial w^{(L)}} = \frac{\partial R_i(\theta^t)}{\partial A^{(L)}} \frac{\partial A^{(L)}}{\partial z^{(L)}} \frac{\partial z^{(L)}}{\partial w^{(L)}}$$

... that's what we call the **chain rule**

Instead of asking how $w^{(L)}$ impacts $R_i(\theta^t)$.

We ask much simpler questions.

For simplicity, let's assume this to be our network:



Let's walk our way backwards:

$$R_i(\theta^t) = -[y_i \log(A^{(L)}) + (1 - y_i) \log(1 - A^{(L)})]$$

$$A^{(L)} = g(z^{(L)})$$

$$z^{(L)} = w^{(L)} A^{(L-1)} + w_0^{(L)}$$

Chain rule allows to find those gradients efficiently.

$$\frac{\partial R_i(\theta^t)}{\partial w^{(L)}} = \frac{\partial R_i(\theta^t)}{\partial A^{(L)}} \cdot \frac{\partial A^{(L)}}{\partial z^{(L)}} \cdot \frac{\partial z^{(L)}}{\partial w^{(L)}}$$

... that's what we call the **chain rule**

?

So, what is the value
of each term?

$$\frac{\partial R_i(\theta^t)}{\partial A^{(L)}}$$



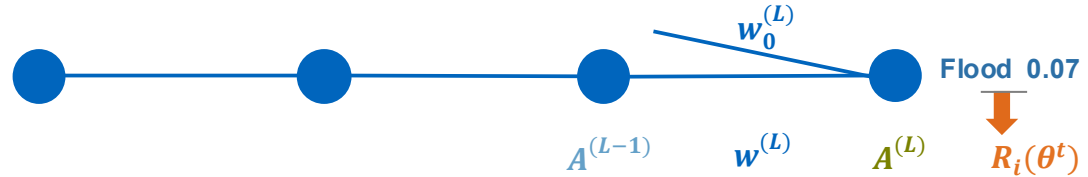
$$\frac{\partial A^{(L)}}{\partial z^{(L)}}$$



$$\frac{\partial z^{(L)}}{\partial w^{(L)}}$$



For simplicity, let's assume this to be our network:



$$R_i(\theta^t) = -[y_i \log(A^{(L)}) + (1 - y_i) \log(1 - A^{(L)})]$$

$$A^{(L)} = g(z^{(L)})$$

$$z^{(L)} = w^{(L)} A^{(L-1)} + w_0^{(L)}$$

?

But what about all
the other
parameters?

$$\nabla R(\theta^t) = \left. \frac{\partial R(\theta)}{\partial \theta} \right|_{\theta=\theta^t}$$

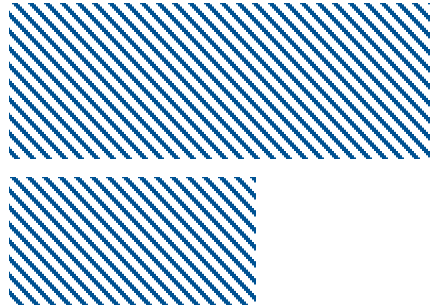
Chain rule allows to find those gradients efficiently.

$$\frac{\partial R_i(\theta^t)}{\partial w^{(L)}} = \frac{\partial R_i(\theta^t)}{\partial A^{(L)}} \cdot \frac{\partial A^{(L)}}{\partial z^{(L)}} \cdot \frac{\partial z^{(L)}}{\partial w^{(L)}}$$

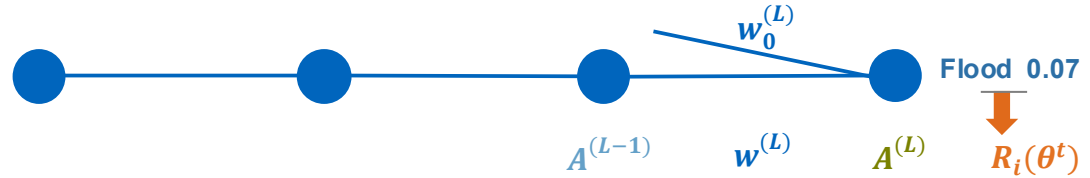


So, what changes
for $w_0^{(L)}$?

$$\frac{\partial R_i(\theta^t)}{\partial w_0^{(L)}}$$

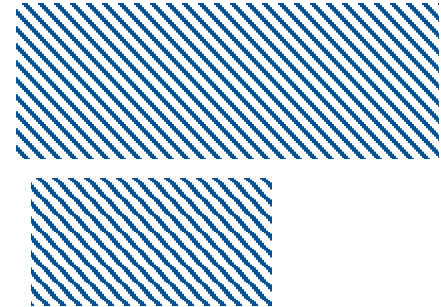


For simplicity, let's assume this to be our network:



What changes
for $A^{(L-1)}$?

$$\frac{\partial R_i(\theta^t)}{\partial A^{(L-1)}}$$



Chain rule allows to find those gradients efficiently.

$$\frac{\partial R_i(\theta^t)}{\partial w^{(L)}} = \frac{\partial R_i(\theta^t)}{\partial A^{(L)}} \cdot \frac{\partial A^{(L)}}{\partial z^{(L)}} \cdot \frac{\partial z^{(L)}}{\partial w^{(L)}}$$

$$\frac{\partial R_i(\theta^t)}{\partial A^{(L-1)}}$$

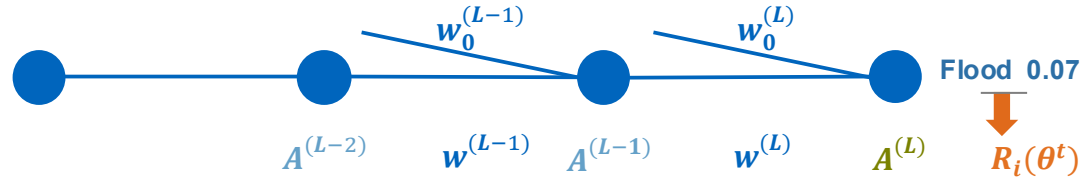
$$\frac{\partial R_i(\theta^t)}{\partial w_0^{(L)}}$$



Lucky shot! How about parameters in earlier layers?

$$\nabla R(\theta^t) = \left. \frac{\partial R(\theta)}{\partial \theta} \right|_{\theta=\theta^t}$$

For simplicity, let's assume this to be our network:

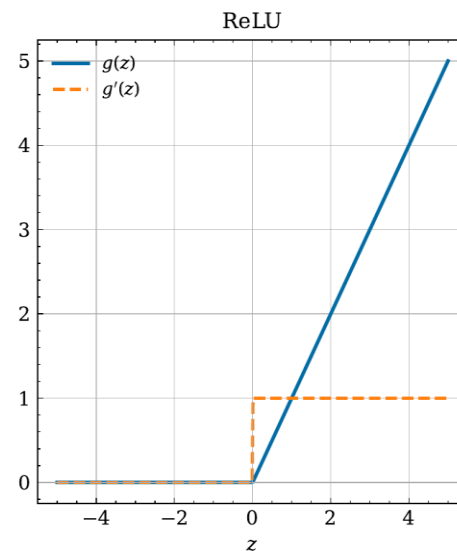
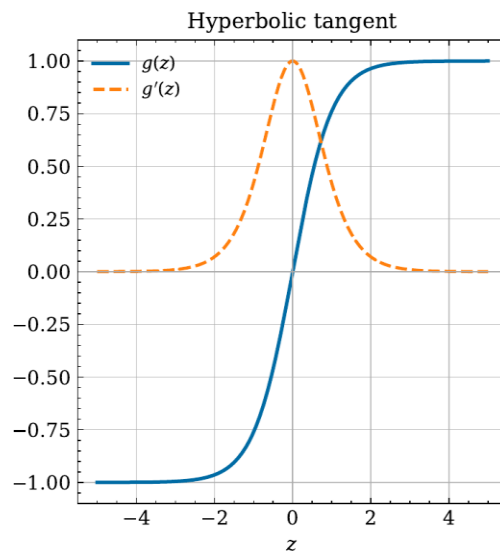
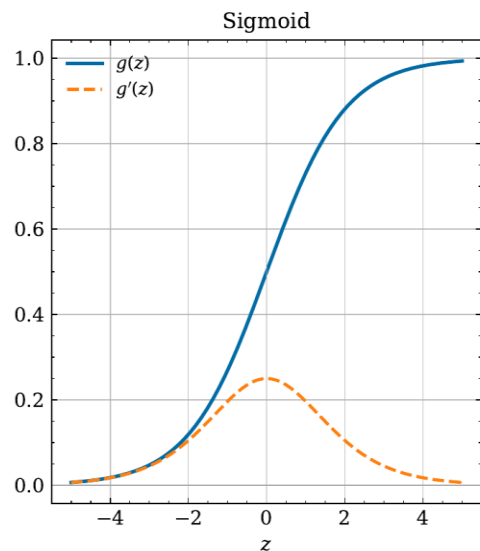


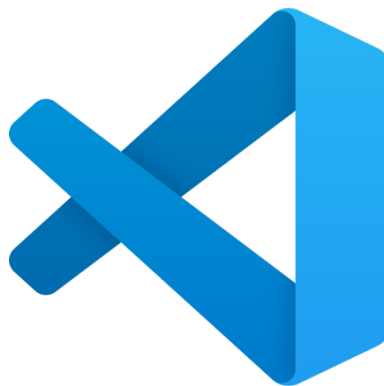
$$\frac{\partial R_i(\theta^t)}{\partial w^{(L-1)}} = \frac{\partial R_i(\theta^t)}{\partial A^{(L)}} \cdot \frac{\partial A^{(L)}}{\partial z^{(L)}} \cdot \frac{\partial z^{(L)}}{\partial A^{(L-1)}} \cdot \frac{\partial A^{(L-1)}}{\partial z^{(L-1)}} \cdot \frac{\partial z^{(L-1)}}{\partial w^{(L-1)}}$$

$$\frac{\partial R_i(\theta^t)}{\partial A^{(L-2)}} \dots$$

$$\frac{\partial R_i(\theta^t)}{\partial w_0^{(L-1)}}$$

Backprop uses the activation derivative $g'(z)$.





- Motivation to Neural Networks
- Network architecture
- Loss and backpropagation
- **Optimization**
- Regularization and tuning

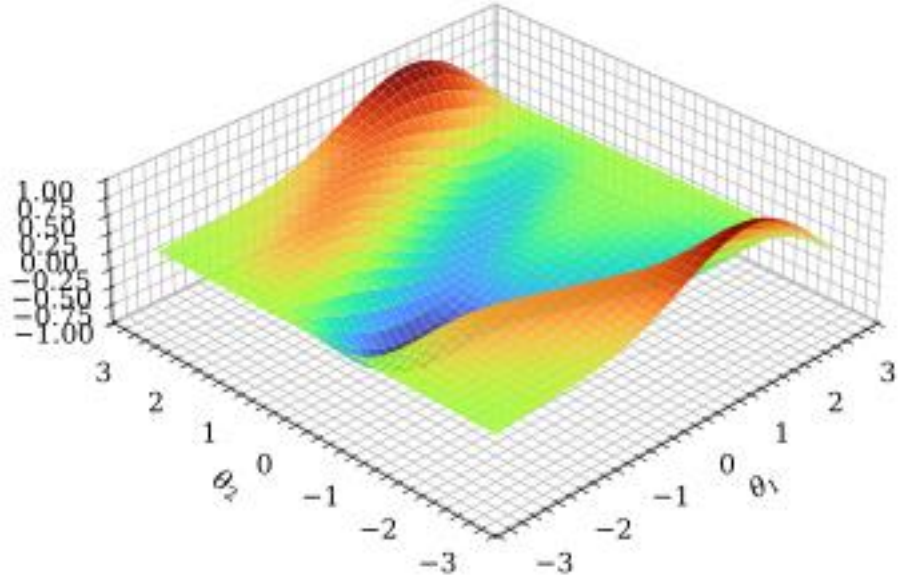
Gradient descent generalizes to higher dimensional spaces.

At the current weight θ^t we read the **tangent**,
the gradient of the loss:

$$\nabla R_i(\theta^t)$$

We step downhill by the **learning rate** ρ , then
move to the next data point:

$$\theta^{t+1} \leftarrow \theta^t - \rho \nabla R_i(\theta^t)$$



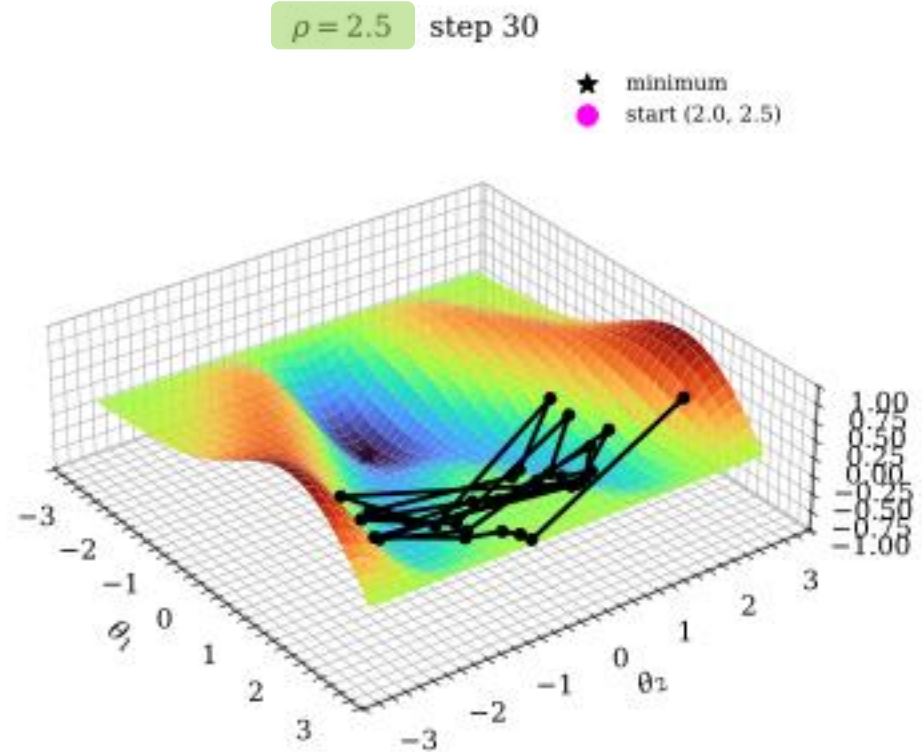
With ρ too big we overshoot and never find the minimum.

At the current weight θ^t we read the **tangent**,
the gradient of the loss:

$$\nabla R_i(\theta^t)$$

We step downhill by the **learning rate ρ** , then
move to the next data point:

$$\theta^{t+1} \leftarrow \theta^t - \rho \nabla R_i(\theta^t)$$



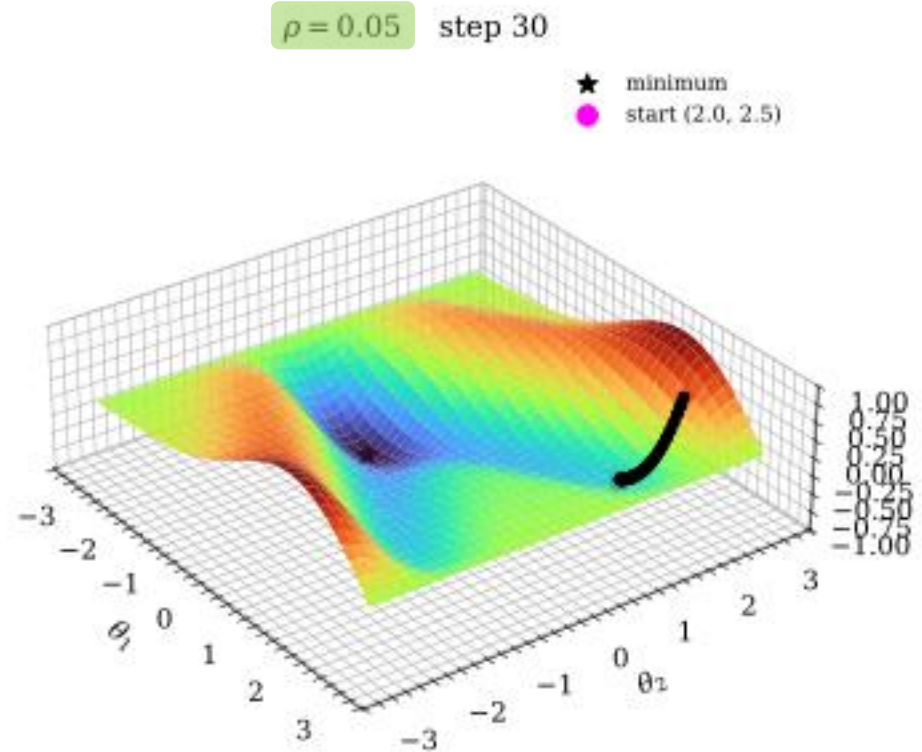
With ρ too small we get stuck in a loss valley.

At the current weight θ^t we read the **tangent**,
the gradient of the loss:

$$\nabla R_i(\theta^t)$$

We step downhill by the **learning rate ρ** , then
move to the next data point:

$$\theta^{t+1} \leftarrow \theta^t - \rho \nabla R_i(\theta^t)$$



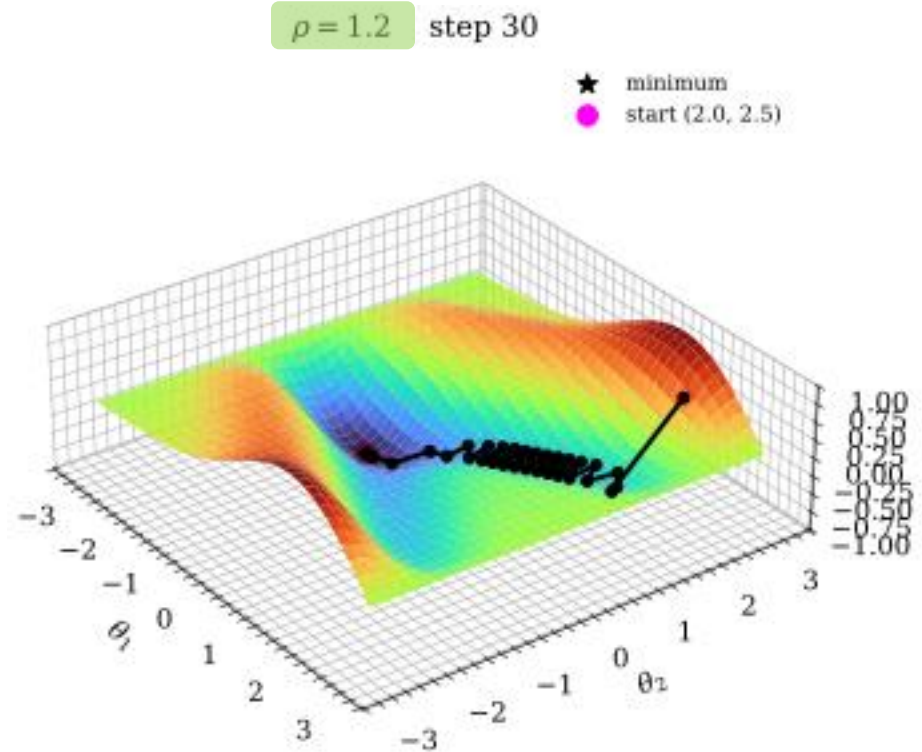
The right learning rate brings us to the minimum.

At the current weight θ^t we read the **tangent**,
the gradient of the loss:

$$\nabla R_i(\theta^t)$$

We step downhill by the **learning rate ρ** , then
move to the next data point:

$$\theta^{t+1} \leftarrow \theta^t - \rho \nabla R_i(\theta^t)$$



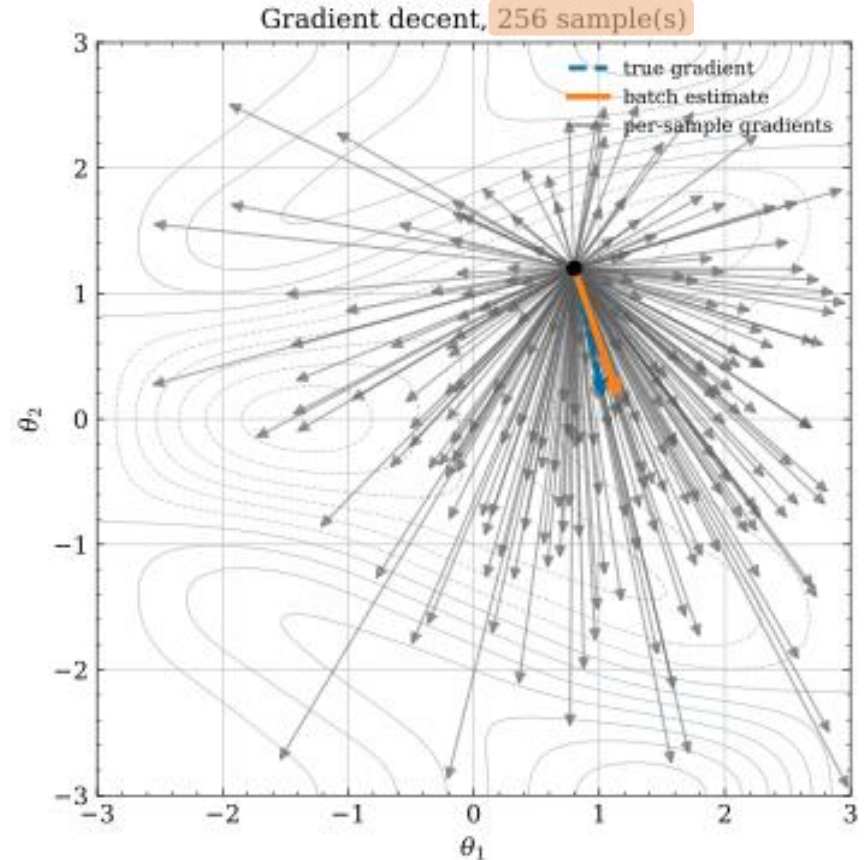
One sample gives a noisy gradient, averaging approaches the true one.

At the current weight θ^t we read the **tangent**, the gradient of the loss:

$$\nabla R_i(\theta^t)$$

We do so for a **mini-batch B** of m samples of our data $\{(x_i, y_i)\}_{i=1}^N$. We step downhill by the learning rate ρ in the direction of the average gradient, then move to the next mini-batch:

$$\theta^{(t+1)} \leftarrow \theta^t - (\rho/m) \sum_{i \in B} \nabla R_i(\theta^t)$$



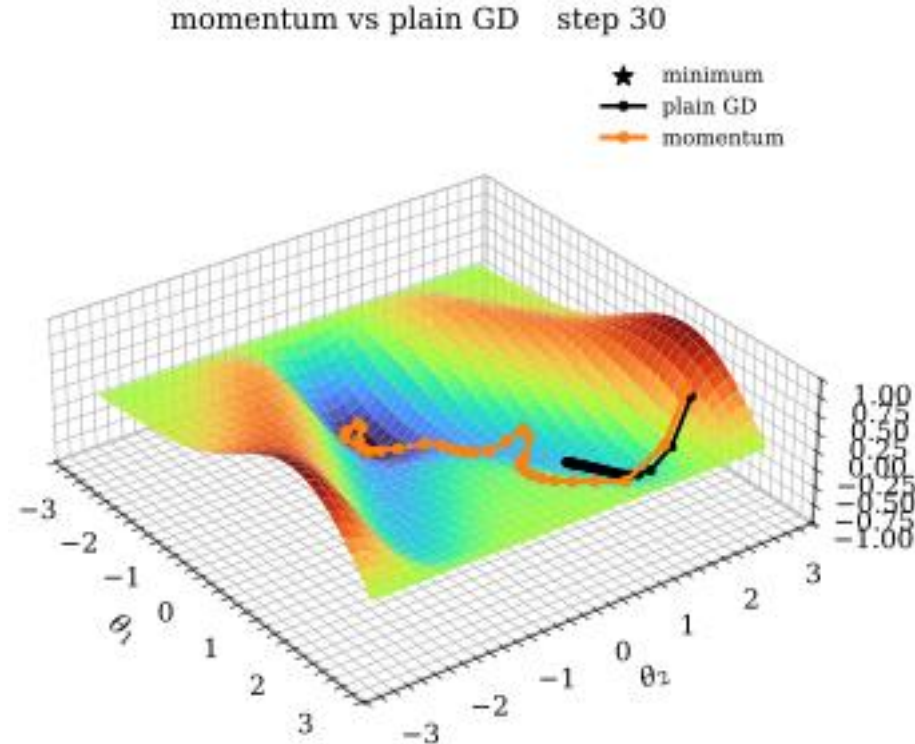
Momentum keeps a running direction.

At the current weight θ^t we read the **tangent**,
the gradient of the loss:

$$\nabla R_i(\theta^t)$$

We fold it into a **running velocity** v that keeps
a fraction γ of the previous direction, then move
along v :

$$\begin{aligned} v &\leftarrow \gamma v - \rho \nabla R_i(\theta^t) \\ \theta^{t+1} &\leftarrow \theta^t + v \end{aligned}$$



Adam: momentum + a per-parameter step.

At the current weight θ^t we read the **tangent**,
the gradient of the loss:

$$\nabla R_i(\theta^t)$$

Adam keeps running averages of the gradient
(**momentum**) and its square (**adaptive scale**).

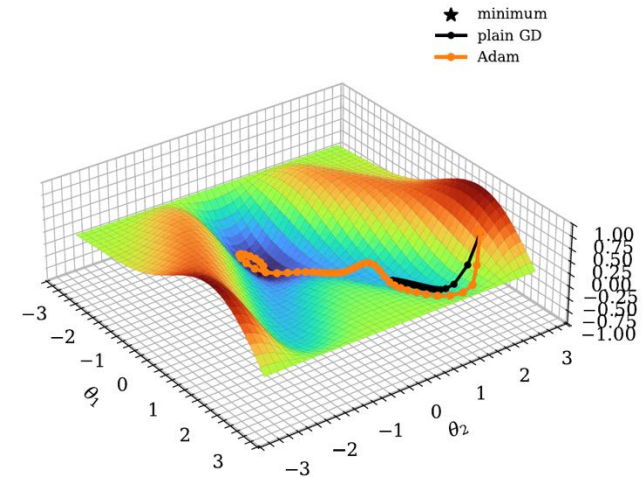
$$m \leftarrow \beta_1 m + (1 - \beta_1) \nabla R_i(\theta^t)$$

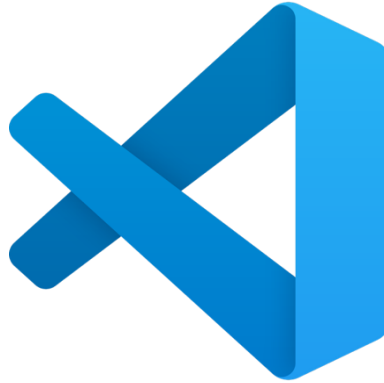
$$v \leftarrow \beta_2 v + (1 - \beta_2) (\nabla R_i(\theta^t))^2$$

We step downhill by the **learning rate ρ** , then
move to the next data point:

$$\theta \leftarrow \theta - \frac{\rho}{\sqrt{\hat{v}} + \epsilon} \hat{m}$$

Adam vs plain GD step 40

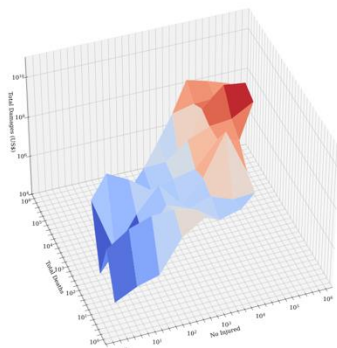




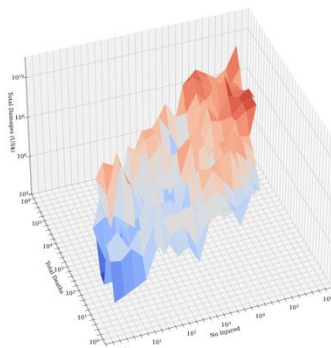
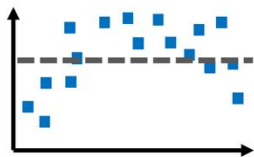
- Motivation to Neural Networks
- Network architecture
- Loss and backpropagation
- Optimization
- Regularization and tuning

Quick rewind: bias variance trade-off.

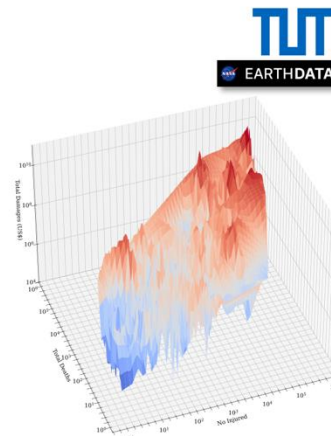
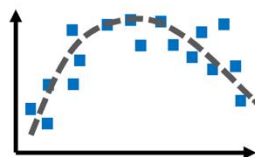
Bias Variance trade-off



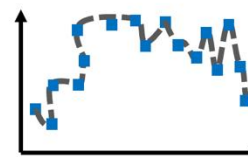
High bias



Low bias, low variance



High variance



Quick rewind: regularization linear regression.

Lasso's L1 penalty can zero coefficients out, doing feature selection.

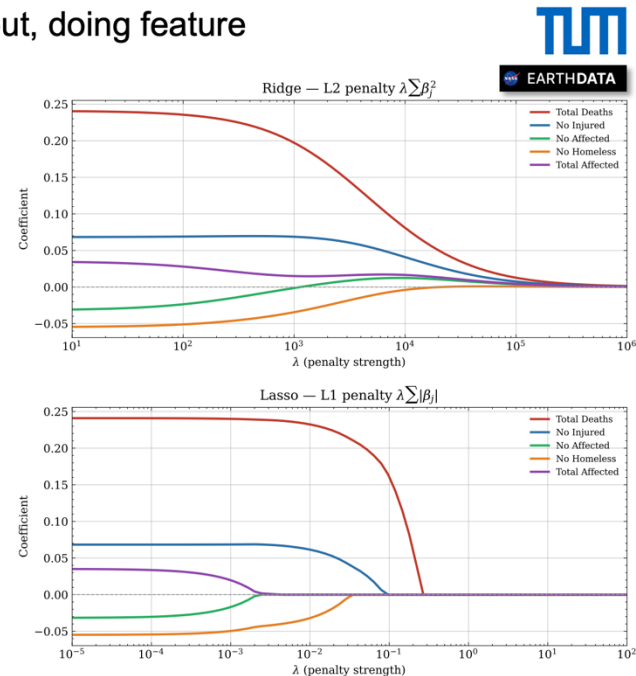
What if we not only want to shrink coefficients but force our model to select a subset of predictors?

Linear regression with this L-1 regularization term is called **Lasso regression**.

$$\min RSS + \lambda \sum_{j=1}^p |\hat{\beta}_j|$$



Why does **L-1 regularization (Lasso)** lead to feature selection?



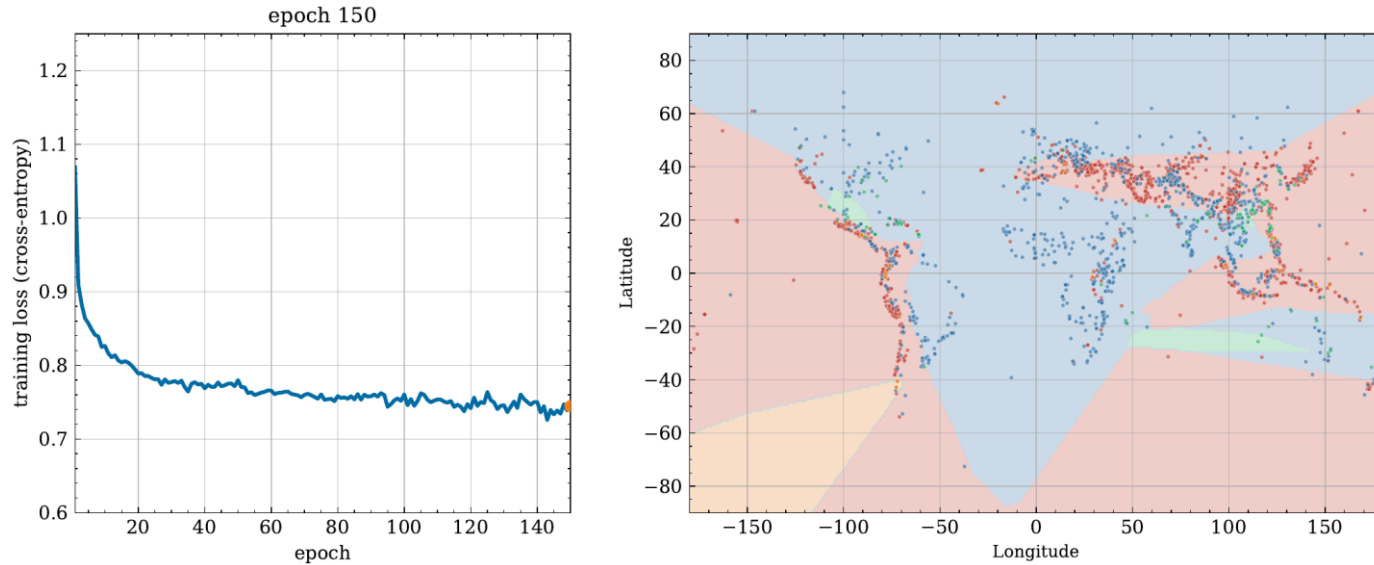
Regularization for neural network loss.

So far we have looked for the parameters θ that minimize the loss R . For classification this loss is the cross-entropy. For regression it is the squared error.

$$R_i(\theta) = \begin{cases} (A^{(L)} - y_i)^2 & \text{regression} \\ - \sum_{m=1}^M y_{i,m} \log(A_m^{(L)}) & \text{classification} \end{cases}$$

Just like in the regression case, we can add a **ridge regularization** on the weights.

Putting it together: watching the classifier train.



Left: training loss falls. Right: decision regions sharpen.